

Extending Anonymity to Authentication Servers in Lightweight Decoupled IoT Authentication:

A Scheme Design and Analysis

MTP Future Work Document

Anup Kulkarni

Roll No. 244101005

Under the Guidance of Dr. Manas Khatua

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Guwahati – 781039, India

June 2026

Contents

1	Introduction and Motivation	3
1.1	Background	3
1.2	Contribution of This Document	3
1.3	Scope	4
2	Related Work	4
2.1	Infrastructure Pseudonymity in Vehicular Networks	4
2.2	Subscriber Identity Concealment in 5G	4
2.3	Anonymity Taxonomy	4
2.4	Anonymity in IoT Authentication Schemes	5
3	Problem Statement: Where ID_{AS} Leaks	5
3.1	Original Scheme Message Flows	5
3.2	Identified Leakage Points	5

4	Extended Scheme: AS Pseudonym Rotation	6
4.1	Design Principles	6
4.2	New Notation	6
4.3	Modified State Tables	8
4.4	GW: AS Pseudonym Initialisation and Epoch Rotation	9
4.5	Modified Phase 1: Enrollment	10
4.6	Modified Phase 2: Authentication	12
4.7	Modified Phase 3: Key Exchange	14
4.8	Dual-State AS Pseudonym Desynchronisation Recovery	16
5	Complete Modified Protocol Summary	17
6	Security Analysis	18
6.1	AS Anonymity Against External Adversary	18
6.2	Epoch-Unlinkability of AS Sessions	18
6.3	AS Identity Secrecy (One-Way Pseudonym)	19
6.4	Accountability: GW Can De-anonymise	19
6.5	Forward Secrecy of AS Pseudonym	19
6.6	Correctness Under Desynchronisation	20
6.7	Preservation of Original Security Properties	20
7	Overhead Analysis	21
7.1	Communication Overhead	21
7.2	Computation Overhead	21
7.3	Storage Overhead	22
8	Epoch Management Policy	22
9	Discussion	23
9.1	Comparison with Device Pseudonym Mechanism	23
9.2	Complete Anonymity: Who Knows Whom	23
9.3	Open Questions for Future Formalisation	24
10	Conclusion	24

1 Introduction and Motivation

1.1 Background

The proposed authentication scheme [1] introduces *device* anonymity into a lightweight, decoupled, distributed authentication framework for multi-hop IoT networks. Devices transmit a rotating pseudonym $PID_{\text{curr}} = H(ID_D || m_{\text{curr}})$ in place of their real identity, so no on-path observer — including the gateway (GW) — learns ID_D . A dual-state mechanism $(PID_{\text{curr}}, PID_{\text{old}}, m_{\text{curr}}, m_{\text{old}})$ at the Authentication Server (AS) handles lost key-exchange packets without re-enrollment.

While device identity is fully shielded, the identity of the *Authentication Server* is not. The current scheme transmits ID_{AS} in plaintext in the AS-to-GW Phase 3 message, and embeds it inside a hash mask used during Phase 2. A passive adversary monitoring the channel can therefore determine:

1. which physical node is currently acting as AS, and
2. which sessions that node authenticated (by correlating the AS-to-GW messages).

This exposes the network topology. In many IoT deployments — battlefield sensor nets, critical infrastructure monitoring, healthcare body-area networks — the identity of authentication infrastructure nodes is itself sensitive. If an adversary can identify the AS, it becomes a high-value target for denial-of-service or physical capture.

1.2 Contribution of This Document

This document proposes **AS Pseudonym Rotation**: a minimal, backward-compatible extension of the proposed scheme that assigns each AS a rotating, epoch-bound pseudonym PID_{AS} managed by the GW. The key properties are:

1. ID_{AS} never appears in plaintext on the open channel.
2. Sessions authenticated by the same AS are unlinkable to a passive observer.
3. The GW (and only the GW) can de-anonymize PID_{AS} to recover ID_{AS} — accountability is preserved.
4. A dual-state AS pseudonym mechanism handles epoch-transition desynchronization, mirroring the device desync mechanism.
5. Communication overhead is limited to one additional 256-bit field (Ψ) in the Phase 3 AS-to-Device message.

1.3 Scope

This document is a *design and feasibility study*. It is not a published paper; figures, protocol descriptions, and security arguments are informal but rigorous. Formal ProVerif verification is deferred as future work.

2 Related Work

2.1 Infrastructure Pseudonymity in Vehicular Networks

The most mature body of work on *server-side* anonymity comes from Vehicular-to-Infrastructure (V2X) systems. IEEE 1609.2 [2] defines a pseudonym certificate framework in which Road-Side Units (RSUs) are assigned short-lived certificates by a Pseudonym Certificate Authority (PCA). Vehicles authenticate to an RSU using the RSU’s current pseudonym certificate; the certificate changes periodically so that long-term vehicle–RSU associations cannot be built by observers. Khodaei et al. [3] study the scalability of this approach and show that GW-coordinated epoch changes with dual-state overlap windows eliminate the need for vehicle-side re-registration during certificate transitions. The AS pseudonym scheme proposed here is structurally analogous: the GW plays the role of the PCA, and the dual-state $(PID_{AS}^{\text{curr}}, PID_{AS}^{\text{old}})$ mechanism mirrors the overlap window.

2.2 Subscriber Identity Concealment in 5G

The 3GPP 5G SA specification (TS 33.501 [4]) introduces the *Subscriber Concealed Identifier* (SUCI), which conceals the long-term Subscriber Permanent Identifier (SUPI) using the home network’s public key. Although SUCI is asymmetrically encrypted rather than hash-based, the functional goal is identical to our PID_{AS} : a one-use alias that allows the network to authenticate the subscriber without transmitting the permanent identity. Importantly, only the Home Network (analogous to our GW) can recover the SUPI from the SUCI. Our construction is lighter: it uses only a shared secret and a collision-resistant hash function, avoiding asymmetric operations that are prohibitive on IoT hardware.

2.3 Anonymity Taxonomy

Pfitzmann and Hansen [5] define three related notions relevant here. *Anonymity*: an agent cannot be identified within a set of subjects. *Unlinkability*: two protocol runs cannot be associated with the same subject. *Pseudonymity*: an agent uses a pseudonym instead of a true identity, with possible de-pseudonymization by an authorised party. The properties achieved by the proposed extension are:

- **AS Anonymity** with respect to external observers (they see only PID_{AS}).

- **Epoch-unlinkability** of AS across sessions: sessions in different epochs are unlinkable even if the observer can link sessions *within* an epoch.
- **Accountable Pseudonymity**: the GW can always de-anonymize PID_{AS} .

2.4 Anonymity in IoT Authentication Schemes

Most existing lightweight IoT authentication schemes protect *user/device* identity. Ali and Ahmed [6] (LAAKA) use temporary identity tokens that change per session. Zhou et al. [7] mask the user ID inside an XOR expression. He et al. [8] use a nonce-refreshed alias. Alruwaili et al. [9] (RAAF-MEC) conceal user identity in a Mobile Edge Computing context. **None of these schemes address the anonymity of the authentication server or infrastructure node itself.**

The proposed scheme of Das and Khatua [1] is the first in this class to address device anonymity through pseudonym rotation; the present document extends it to cover AS anonymity, achieving end-to-end anonymity for all entities except the GW itself.

3 Problem Statement: Where ID_{AS} Leaks

3.1 Original Scheme Message Flows

Recall the four phases of the proposed scheme:

1. **Enrollment** (D to AS via GW): GW assigns AS to device D and communicates ID_{AS} to D. AS stores $(y_D, ID_D, PID_{curr}, PID_{old}, m_{curr}, m_{old}, K_{GW-D}, K_{GW-AS}, T_{Acc})$. D stores $(y_D, c_D, m_{curr}, PID_{curr}, ID_{AS})$.
2. **Authentication** (D to AS): D computes $Y_{AS-D}^H = H(y_D) \oplus H(R_D || m_{curr} || PID_{curr} || ID_{AS} || ts_1)$ and transmits $(PID_{curr}, Y_{AS-D}^H, ts_1)$ to AS.
3. **Key Exchange** (AS to D; AS to GW):
 - AS→D: (m_H, ts_2) where $m_H = m_{new} \oplus H(Y_D^H || m_{curr} || R'_D || ID_{AS} || PID_{curr} || ts_2)$
 - AS→GW: $(PID_{new}, \underline{ID_{AS}}, auth_token_D)$ where $auth_token_D = SE(K_{GW-AS}, (K_{GW-D} || ID_{AS}))$
4. **Data Communication** (D to GW): uses session key SK .

3.2 Identified Leakage Points

Leakage L1 (Plaintext transmission): The AS-to-GW Phase 3 message contains ID_{AS} in *cleartext*. Any node on the path between AS and GW sees the real identity of the AS.

Leakage L2 (Hash embedding): ID_{AS} appears inside the hash mask in both Phase 2 and Phase 3. While the hash itself is one-way, the value is used as a *static* component across all sessions. An adversary who knows ID_{AS} (e.g., from L1 in an earlier session) can verify this association in future sessions.

Leakage L3 (Enrollment distribution): ID_{AS} is transmitted from GW to D during enrollment. If the enrollment channel is not perfectly confidential, ID_{AS} leaks at registration time.

Leakage L1 is the most damaging: it places ID_{AS} in the open in *every authenticated session*, allowing passive traffic analysis to construct a complete map of which nodes serve as AS over time.

4 Extended Scheme: AS Pseudonym Rotation

4.1 Design Principles

The extension is governed by four principles:

1. **Minimal change.** Only the fields involving ID_{AS} are modified. The PUF-based secret derivation, hash accumulator, and device-side pseudonym mechanism are unchanged.
2. **GW-managed epochs.** The GW is the single point that knows the true identity of every AS. It generates, distributes, and tracks AS pseudonyms.
3. **Dual-state recovery.** An AS pseudonym epoch change between enrollment and first authentication would desync the device's stored PID_{AS} ; dual-state storage at AS allows recovery without re-enrollment.
4. **No new cryptographic primitives.** Only hash functions and XOR operations are used, consistent with the original scheme.

4.2 New Notation

Table 1 summarises the symbols introduced by this extension.

Table 1: New symbols introduced by the AS pseudonym extension.

Symbol	Meaning
$\eta_{\text{curr}}, \eta_{\text{old}}$	GW-generated nonces for the current and previous AS pseudonym epoch
$PID_{AS}^{\text{curr}} = H(ID_{AS} \eta_{\text{curr}})$	Current AS pseudonym (computed and known only to GW and AS)
$PID_{AS}^{\text{old}} = H(ID_{AS} \eta_{\text{old}})$	Previous AS pseudonym (for dual-state desync recovery)
Ψ	Masked AS pseudonym update field sent in Phase 3 AS→D message; $\Psi = PID_{AS}^{\text{curr}} \oplus H(K_{GW-D} ts_2)$
epoch_{AS}	AS pseudonym epoch identifier (counter or timestamp, maintained by GW)
ρ_{AS}	Long-term shared secret between GW and AS (used in key derivation)

For quick reference, Figure 1 gives a plain-language glossary of every symbol used in the protocol diagrams that follow; the shaded rows are the new elements introduced for AS anonymity.

Notation used in the AS-Anonymity Scheme

Symbol	Meaning (plain language)
ID_D	Real identity of the device (kept secret, never sent on air)
ID_{AS}	Real identity of the Authentication Server (kept secret, never sent)
$PID = H(ID_D \parallel m_{curr})$	Device pseudonym: a disposable alias used on air in place of the real device ID
PID_{curr}, PID_{old}	Current and previous device pseudonyms (the pair lets a lost packet be recovered)
$PID_{AS}^{curr}, PID_{AS}^{old}$	Current and previous AS pseudonyms: disposable alias for the AS (NEW)
η_{curr}, η_{old}	Secret random numbers the gateway uses to build the AS pseudonym each epoch (NEW)
Ψ	Hidden update that quietly tells the device the AS's new pseudonym (NEW)
$m_{curr}, m_{old}, m_{new}$	Session random numbers; they refresh the pseudonyms after each round
R_D	Device PUF response: a hardware secret regenerated on the fly, never stored
y_D, Φ_{AS-D}	Device secret and its PUF-masked form held at the AS
T_{Acc}	One-way accumulator the AS uses to check group membership
K_{GW-D}	Session key shared between the gateway and the device
K_{GW-AS}	Long-term key shared between the gateway and the AS
$auth_token_D$	Encrypted token the AS gives the gateway to hand over the device's session
SE, SD	Symmetric-key encryption and decryption
ts_1, ts_2, ts_{auth}	Timestamps used to check that a message is fresh (not replayed)
$H(\cdot), \oplus, \parallel$	One-way hash function, bitwise XOR, and concatenation

Shaded rows are new elements introduced for AS anonymity.

Figure 1: Plain-language notation for the AS-anonymity scheme. Shaded rows mark the symbols added by this extension.

4.3 Modified State Tables

Tables 2–4 show the complete state stored by each entity after the extension.

Table 2: Extended GW state (additions highlighted).

Entity: GW	Stored per AS
ID_{AS}	Real identity of AS (GW-local, never transmitted)
K_{GW-AS}	Long-term shared key with AS
PID_{AS}^{curr}	Current AS pseudonym
PID_{AS}^{old}	Previous AS pseudonym
η_{curr}, η_{old}	Current and previous epoch nonces

Table 3: Extended AS state (additions highlighted).

Entity: AS	Stored
<i>Per-device state (unchanged from original scheme):</i>	
$y_D, ID_D, m_{\text{curr}}, m_{\text{old}}, PID_{\text{curr}}, PID_{\text{old}}, K_{GW-D}, K_{GW-AS}, T_{Acc}$	(same as before)
<i>Own identity state:</i>	
ID_{AS}	Real identity (local only)
PID_{AS}^{curr}	Own current pseudonym (received from GW)
PID_{AS}^{old}	Own previous pseudonym (for reference in verification)

Table 4: Extended Device state (additions highlighted).

Entity: Device D	Stored
$y_D, c_D, m_{\text{curr}}, PID_{\text{curr}}$	(same as before)
PID_{AS}^{curr}	Current pseudonym of assigned AS (replaces ID_{AS})

4.4 GW: AS Pseudonym Initialisation and Epoch Rotation

The GW manages AS pseudonym epochs independently of the session protocol. Epoch rotation is triggered by a time-based policy (e.g., every 24 hours) or a count-based policy (every N sessions).

Pseudonym Initialisation (at system bootstrap, per AS):

GW.1 Generate fresh random nonce $\eta^{(0)}$.

GW.2 Compute $PID_{AS}^{(0)} = H(ID_{AS} || \eta^{(0)})$.

GW.3 Transmit $PID_{AS}^{(0)}$ to AS over the existing secure channel (the same channel used for K_{GW-AS} setup).

GW.4 Store $(PID_{AS}^{\text{curr}} \leftarrow PID_{AS}^{(0)}, PID_{AS}^{\text{old}} \leftarrow \perp, \eta_{\text{curr}} \leftarrow \eta^{(0)}, \eta_{\text{old}} \leftarrow \perp)$.

GW.5 AS stores $(PID_{AS}^{\text{curr}} \leftarrow PID_{AS}^{(0)}, PID_{AS}^{\text{old}} \leftarrow \perp)$.

Epoch Rotation (when epoch boundary is reached):

GW.1 Generate fresh random nonce $\eta^{(k)}$.

GW.2 Compute $PID_{AS}^{(k)} = H(ID_{AS} || \eta^{(k)})$.

GW.3 Transmit $PID_{AS}^{(k)}$ to AS over the secure channel.

GW.4 Update GW state: $PID_{AS}^{old} \leftarrow PID_{AS}^{curr}$, $PID_{AS}^{curr} \leftarrow PID_{AS}^{(k)}$, $\eta_{old} \leftarrow \eta_{curr}$, $\eta_{curr} \leftarrow \eta^{(k)}$.

GW.5 AS updates: $PID_{AS}^{old} \leftarrow PID_{AS}^{curr}$, $PID_{AS}^{curr} \leftarrow PID_{AS}^{(k)}$.

Invariant: At any moment, GW and AS agree on $(PID_{AS}^{curr}, PID_{AS}^{old})$. A device enrolled during epoch $k-1$ stores $PID_{AS}^{(k-1)} = PID_{AS}^{old}$ after epoch k begins; the dual-state mechanism (Section 4.8) handles this transparently.

4.5 Modified Phase 1: Enrollment

The enrollment phase changes only in what the GW communicates to the device about its assigned AS.

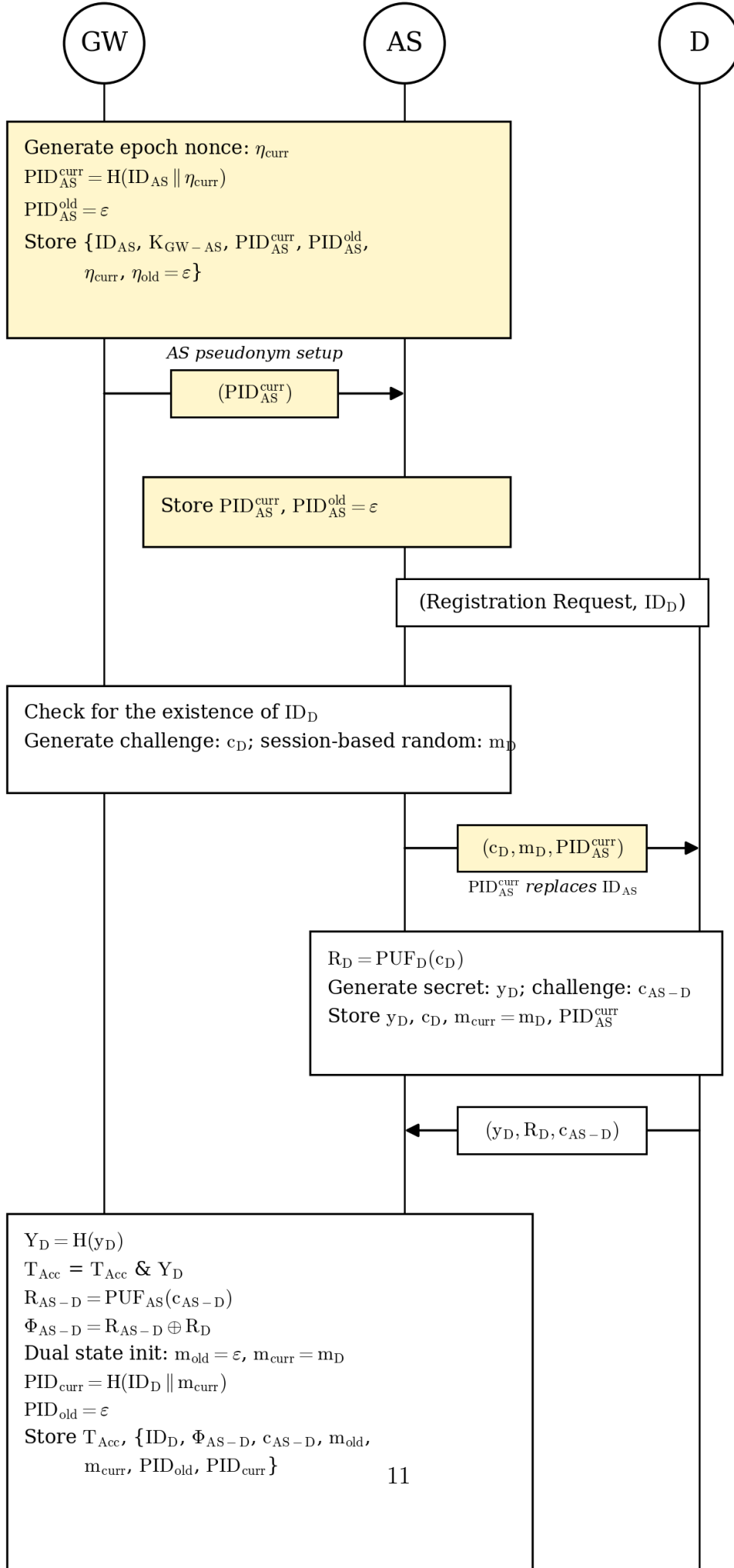
Modified step (GW to D):

Old: GW sends ID_{AS} and the AS's network address to D.

New: GW sends PID_{AS}^{curr} and the AS's network address to D.

D stores PID_{AS}^{curr} in place of ID_{AS} . D never learns the real identity of its assigned AS.

All other enrollment steps (PUF challenge storage, hash accumulator setup, y_D derivation, AS state initialisation) are unchanged. Figure 2 shows the full modified enrollment message flow, with the changed steps shaded.



4.6 Modified Phase 2: Authentication

D computes:

$$Y_{AS-D}^H = H(y_D) \oplus H(R_D || m_{\text{curr}} || PID_{\text{curr}} || \underline{PID_{AS}^{\text{curr}}} || ts_1) \quad (\text{P2-D})$$

(Previously: $H(R_D || m_{\text{curr}} || PID_{\text{curr}} || \underline{ID_{AS}} || ts_1)$.)

D transmits $(PID_{\text{curr}}, Y_{AS-D}^H, ts_1)$ to AS. **Message format unchanged.**

AS verifies: The original scheme tries two combinations of $(m_{\text{curr}}, m_{\text{old}})$ to recover $H(y_D)$ and check membership in T_{Acc} . The extended scheme must now also try two values of PID_{AS} , giving four combinations.

AS iterates over:

$$\mathcal{V} = \{(m_{\text{curr}}, PID_{AS}^{\text{curr}}), (m_{\text{curr}}, PID_{AS}^{\text{old}}), (m_{\text{old}}, PID_{AS}^{\text{curr}}), (m_{\text{old}}, PID_{AS}^{\text{old}})\}$$

in that order (most likely first). For each $(m_{\text{try}}, PID_{AS}^{\text{try}})$:

$$Y_{\text{try}} = Y_{AS-D}^H \oplus H(R'_D || m_{\text{try}} || PID_{\text{curr}} || PID_{AS}^{\text{try}} || ts_1) \quad (\text{P2-AS1})$$

$$\text{Accept if } T_{Acc} \& Y_{\text{try}} = T_{Acc} \quad (\text{P2-AS2})$$

where R'_D is the PUF response AS holds for device D (derived during enrollment). On acceptance, AS records the successful $(m_D^{\text{used}}, PID_{AS}^{\text{used}})$ for use in Phase 3.

Complexity note: In the common case (no desync), only the first combination succeeds, so there is no overhead. In the worst case (both device nonce and AS epoch are stale), all four are tried: three extra hash+XOR operations. Given that IoT authentication is compute-limited by the network round-trip, not local hashing, this is negligible.

Figure 3 shows the full modified authentication phase, including the four-way dual-state verification performed by the AS.

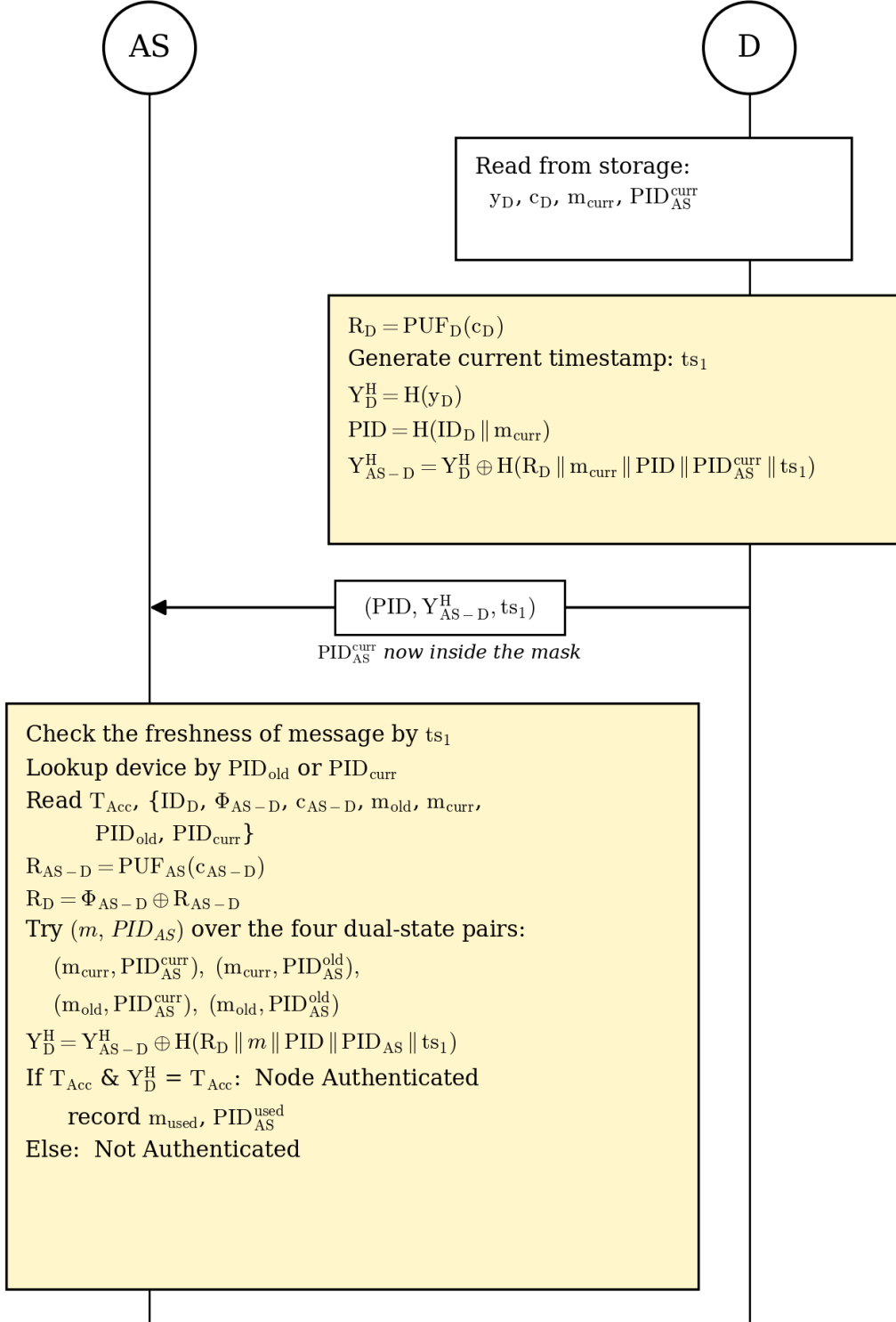


Figure 3: Modified authentication phase. The device folds PID_{AS}^{curr} into the verification mask, and the AS searches the four dual-state (m, PID_{AS}) pairs until the accumulator check passes (shaded steps are new).

4.7 Modified Phase 3: Key Exchange

This is the primary modification. Figure 4 gives the detailed message flow with all computations, and the schematic in Figure 5 highlights the boldface changes from the original scheme.

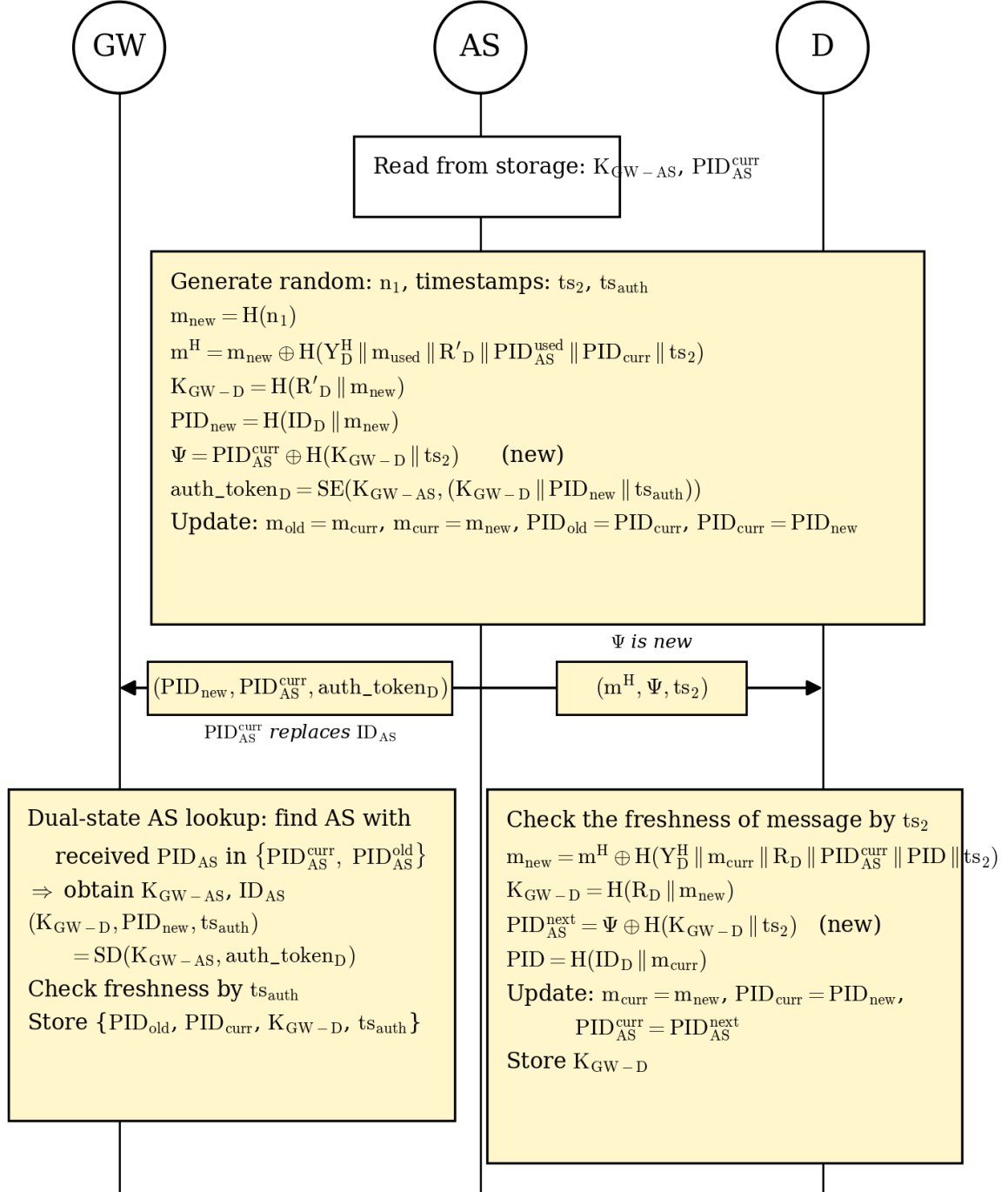


Figure 4: Modified key exchange phase (detailed). AS adds the masked field Ψ to refresh the device's stored AS pseudonym, and sends PID_{AS}^{curr} (not ID_{AS}) to the GW; the GW resolves the AS via a dual-state pseudonym lookup (shaded steps are new).

AS Computes

$$m_{\text{new}} = H(n_1) \quad (\text{fresh nonce } n_1 \text{ generated by AS}) \quad (\text{P3-AS1})$$

$$K_{GW-D} = H(R'_D || m_{\text{new}}) \quad (\text{P3-AS2})$$

$$PID_{\text{new}} = H(ID_D || m_{\text{new}}) \quad (\text{P3-AS3})$$

$$m_H = m_{\text{new}} \oplus H(Y_D^H || m_D^{\text{used}} || R'_D || \underline{PID_{AS}^{\text{used}}} || PID_{\text{curr}} || ts_2) \quad (\text{P3-AS4})$$

$$\Psi = PID_{AS}^{\text{curr}} \oplus H(K_{GW-D} || ts_2) \quad (\text{P3-AS5})$$

$$auth_token_D = SE(K_{GW-AS}, (K_{GW-D} || PID_{\text{new}} || ts_{auth})) \quad (\text{P3-AS6})$$

Note: m_H uses PID_{AS}^{used} (the PID_{AS} value D actually had), while Ψ always carries PID_{AS}^{curr} (the current epoch value, which may or may not differ).

$$\text{AS} \rightarrow \text{D: } (m_H, \Psi, ts_2) \quad (\text{added } \Psi)$$

$$\text{AS} \rightarrow \text{GW: } (PID_{\text{new}}, \underline{PID_{AS}^{\text{curr}}}, auth_token_D) \quad (ID_{AS} \text{ replaced by } PID_{AS}^{\text{curr}})$$

D Processes AS→D

$$m_{\text{new}} = m_H \oplus H(Y_D^H || m_{\text{curr}} || R_D || \underline{PID_{AS}^{\text{curr}}} || PID_{\text{curr}} || ts_2) \quad (\text{P3-D1})$$

$$K_{GW-D} = H(R_D || m_{\text{new}}) \quad (\text{P3-D2})$$

$$PID_{AS}^{\text{next}} = \Psi \oplus H(K_{GW-D} || ts_2) \quad (\text{P3-D3})$$

$$PID_{\text{new}} = H(ID_D || m_{\text{new}}) \quad (\text{P3-D4})$$

D updates state: $m_{\text{curr}} \leftarrow m_{\text{new}}, PID_{\text{curr}} \leftarrow PID_{\text{new}}, PID_{AS}^{\text{curr}} \leftarrow PID_{AS}^{\text{next}}$.

AS pseudonym consistency: If AS used $PID_{AS}^{\text{used}} = PID_{AS}^{\text{old}}$ in m_H (because D had old PID_{AS}), D uses its stored $PID_{AS}^{\text{curr}} = PID_{AS}^{\text{old}}$ in Eq. (P3-D1) and correctly recovers m_{new} . Then Eq. (P3-D3) un.masks $\Psi = PID_{AS}^{\text{currnew}} \oplus H(K_{GW-D} || ts_2)$ to give D the *new* current pseudonym. After step D3, D has the correct PID_{AS}^{curr} for the next session.

GW Processes AS→GW

GW.1 Receive $(PID_{\text{new}}, PID_{AS}^{\text{rcv}}, auth_token_D)$.

GW.2 Dual-state AS lookup: Find AS such that $PID_{AS}^{\text{rcv}} \in \{PID_{AS}^{\text{curr}}, PID_{AS}^{\text{old}}\}$ for that AS. This yields K_{GW-AS} and ID_{AS} .

GW.3 Decrypt: $(K_{GW-D}, PID'_{\text{new}}, ts_{auth}) \leftarrow SE^{-1}(K_{GW-AS}, auth_token_D)$.

GW.4 Verify $PID_{\text{new}} = PID'_{\text{new}}$ and timestamp freshness.

GW.5 Store device session: $(D \mapsto (PID_{\text{new}}, K_{GW-D}))$.

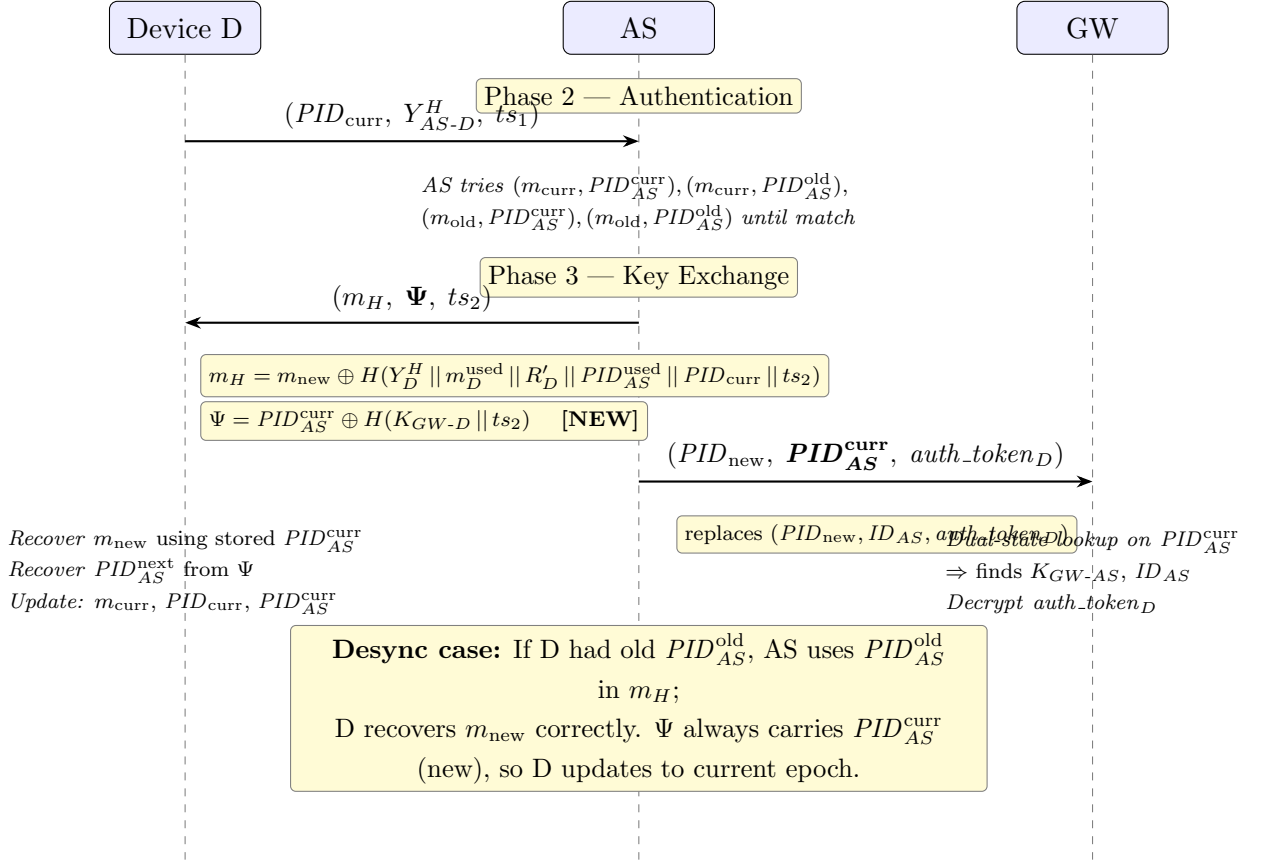


Figure 5: Modified Phase 3 key exchange. Boldface items are changed from the original scheme. GW lookup is now on PID_{AS}^{curr} (not ID_{AS}); D receives Ψ to refresh its stored AS pseudonym.

4.8 Dual-State AS Pseudonym Desynchronisation Recovery

The dual-state mechanism for the AS pseudonym mirrors the existing device-pseudonym mechanism. Table 5 enumerates all desync scenarios.

Table 5: Desynchronisation scenarios and recovery paths.

Scenario	Root cause	Recovery
D has m_{old} , current PID_{AS}^{curr}	Phase 3 AS→D lost; D did not update m_{curr}	AS verifies with $(m_{\text{old}}, PID_{AS}^{\text{curr}})$; uses same in m_H ; Ψ confirms PID_{AS}^{curr}
D has m_{curr} , stale PID_{AS}^{old}	Epoch rotated after enrollment, before first auth	AS verifies with $(m_{\text{curr}}, PID_{AS}^{\text{old}})$; uses PID_{AS}^{old} in m_H ; Ψ delivers PID_{AS}^{curr} (new) to D
D has m_{old} , stale PID_{AS}^{old}	Both: lost Phase 3 packet AND epoch rotation	AS verifies with $(m_{\text{old}}, PID_{AS}^{\text{old}})$; Ψ delivers PID_{AS}^{curr}
D has m_{curr} , current PID_{AS}^{curr}	Normal (no desync)	AS verifies with $(m_{\text{curr}}, PID_{AS}^{\text{curr}})$ (first try)

Epoch window constraint: The GW must maintain $(PID_{AS}^{\text{curr}}, PID_{AS}^{\text{old}})$ for each AS, covering exactly two consecutive epochs. Devices that have been silent for more than two epochs (i.e., missed two successive epoch rotations) cannot recover via this mechanism and require re-enrollment. This is acceptable since the epoch period should be set much longer than the expected maximum idle period for any device.

5 Complete Modified Protocol Summary

For reference, Table 6 summarises every message changed by the extension.

Table 6: Per-message diff between original and extended scheme.

Message	Original	Extended
GW→D (enrollment)	$(ID_{AS}, addr_{AS}, \dots)$	$(PID_{AS}^{\text{curr}}, addr_{AS}, \dots)$
D→AS (Phase 2 mask)	$H(R_D m_{\text{curr}} PID_{\text{curr}} ID_{AS} ts_1)$ (embedded in Y_{AS-D}^H ; wire format unchanged)	$H(R_D m_{\text{curr}} PID_{\text{curr}} PID_{AS}^{\text{curr}} ts_1)$
AS→D (Phase 3)	(m_H, ts_2)	(m_H, Ψ, ts_2) +256 bit
AS→GW (Phase 3)	$(PID_{\text{new}}, \mathbf{ID}_{AS}, auth_token_D)$	$(PID_{\text{new}}, \mathbf{PID}_{AS}^{\text{curr}}, auth_token_D)$
Inside $auth_token_D$	$SE(K_{GW-AS}, (K_{GW-D} ID_{AS} PSE(K_{GW-D}, (K_{GW-D} PID_{\text{new}} ts)))$ (removing ID_{AS} from ciphertext; GW identifies AS via PID_{AS}^{curr} lookup)	

6 Security Analysis

6.1 AS Anonymity Against External Adversary

Property 6.1 (AS Anonymity). *A passive adversary observing all messages on all links cannot determine the real identity ID_{AS} of the authentication server from the observed ciphertext.*

Argument. Every wire occurrence of the AS identity is either:

1. Replaced by $PID_{AS}^{\text{curr}} = H(ID_{AS} || \eta_{\text{curr}})$ where η_{curr} is a GW-secret nonce. Recovering ID_{AS} from PID_{AS}^{curr} requires inverting H , which is computationally hard by the collision-resistance assumption.
2. Inside the symmetric ciphertext $auth_token_D$ encrypted under K_{GW-AS} (now, only $K_{GW-D}, PID_{\text{new}}, ts_{\text{auth}}$ are inside — ID_{AS} has been removed from the ciphertext payload).
3. Used only as a local variable inside AS and GW, never transmitted.

Therefore no wire-level observation yields ID_{AS} . □

6.2 Epoch-Unlinkability of AS Sessions

Property 6.2 (AS Epoch Unlinkability). *Given two sessions S_i and S_j from different epochs k and k' ($k \neq k'$), a computationally bounded adversary cannot determine that*

they involved the same AS with probability better than $\frac{1}{2} + \epsilon$ where ϵ is negligible.

Argument. $PID_{AS}^{(k)} = H(ID_{AS} || \eta^{(k)})$ and $PID_{AS}^{(k')} = H(ID_{AS} || \eta^{(k')})$. Since $\eta^{(k)}$ and $\eta^{(k')}$ are independently drawn uniform random nonces, the two pseudonyms are computationally independent. An adversary distinguishing two sessions from the same AS must link two independently uniform outputs of H , which is infeasible under the PRF assumption on H . $\square$

Intra-epoch linkability: Sessions within the same epoch use the same PID_{AS}^{curr} , so they are linkable to the same AS within that epoch. This is an inherent trade-off: epochs must be chosen short enough that linkability within an epoch is acceptable, but long enough that epoch-rotation overhead is tolerable. A 24-hour epoch with a busy AS authenticating 1000 devices per day means at most 1000 sessions are linkable — the same as in any pseudonym-based system with a finite refresh period.

6.3 AS Identity Secrecy (One-Way Pseudonym)

Property 6.3 (One-Way Pseudonym). *Given PID_{AS}^{curr} , no PPT adversary can compute ID_{AS} without knowing η_{curr} .*

This follows directly from the one-wayness of H . The adversary sees only PID_{AS}^{curr} ; η_{curr} is generated fresh per epoch and stored only at the GW (never transmitted).

6.4 Accountability: GW Can De-anonymise

The GW stores $(ID_{AS}, PID_{AS}^{\text{curr}}, PID_{AS}^{\text{old}}, \eta_{\text{curr}}, \eta_{\text{old}})$ for each AS. Given any PID_{AS} value from an observed message, the GW can:

1. Look up PID_{AS} in its table to recover ID_{AS} .
2. Confirm: $PID_{AS} \stackrel{?}{=} H(ID_{AS} || \eta_{\text{curr}})$ or $H(ID_{AS} || \eta_{\text{old}})$.

This is exact, efficient, and requires no cryptographic inversion. The GW can always hold the real AS accountable for any session.

6.5 Forward Secrecy of AS Pseudonym

If a past epoch nonce η_{old} is compromised (e.g., by storage leakage at the GW), the adversary can recompute PID_{AS}^{old} for the affected AS. However:

- Future epochs use fresh, independently generated η , so future PID_{AS} values remain secure.

- Past sessions where PID_{AS}^{old} was used are re-linkable, but no session key SK or device identity ID_D is recovered (those are protected by separate mechanisms in the original scheme).

This is the standard forward-secrecy guarantee of epoch-based pseudonym systems: compromise does not cascade to future epochs.

6.6 Correctness Under Desynchronisation

Theorem 6.1 (Recovery Completeness). *Assuming the AS maintains dual-state $(PID_{AS}^{\text{curr}}, PID_{AS}^{\text{old}})$ and epochs rotate at most once between two consecutive authentication attempts by any device D , the verification in Section 4.6 always succeeds.*

Proof. By case analysis on Table 5: all four combinations of $(m_D^{\text{used}} \in \{m_{\text{curr}}, m_{\text{old}}\}, PID_{AS}^{\text{used}} \in \{PID_{AS}^{\text{curr}}, PID_{AS}^{\text{old}}\})$ are tried. Since the epoch rotates at most once between D 's last successful authentication (when it updated its state) and the current attempt, D 's stored PID_{AS} is at most one epoch stale, i.e., it equals PID_{AS}^{old} . Similarly, m_D^{used} is at most one state behind. Hence the correct combination is always in $\mathcal{V}$, and verification succeeds. $\square$

6.7 Preservation of Original Security Properties

The extension does not alter:

- The PUF-based secret derivation (device secret R_D is never stored or transmitted).
- The hash accumulator membership test (T_{Acc}).
- The device pseudonym rotation mechanism $(PID_{\text{curr}}, PID_{\text{old}})$.
- The session key derivation (SK, K_{GW-D}) .
- The ProVerif-verified properties Q1–Q14 of the original scheme: mutual authentication correspondences, session key secrecy, device anonymity (ID_D), and forward secrecy of m_{new} .

Property Q15 (Device ID anonymity) continues to hold; the extension adds a new property: AS ID anonymity.

7 Overhead Analysis

7.1 Communication Overhead

Table 7: Communication overhead per authentication round (hash output = 256 bits; SE block = 256 bits).

Message	Original (bits)	Extended (bits)	Change
D→AS (Phase 2)	768	768	None (mask change internal)
AS→D (Phase 3)	512	768	+256 (new Ψ field)
AS→GW (Phase 3)	768	768	None (PID_{AS}^{curr} same width as ID_{AS})
Total per round	2048	2304	+256 bits (+12.5%)

The overhead is modest: one additional 256-bit field Ψ in the AS-to-Device message. Out-of-band epoch rotation messages (GW to AS) occur once per epoch and are negligible amortised over the epoch’s session count.

7.2 Computation Overhead

Table 8: Hash (SHA-256 equivalent) and XOR operations added per round.

Entity	Extra ops (worst case)	Description
Device D (Phase 2)	0H + 0X	Mask uses PID_{AS}^{curr} instead of ID_{AS}
Device D (Phase 3)	+1H + 1X	Recover PID_{AS}^{next} from Ψ
AS (Phase 2 verify)	+3H + 3X	Up to 3 extra verification attempts
AS (Phase 3 send)	+1H + 1X	Compute Ψ
GW (Phase 3)	+0–1H	Dual-state lookup (hash compare)
Total (worst)	+6H + 5X	6 extra SHA-256, 5 XOR
Total (normal)	+2H + 2X	No desync

On Raspberry Pi 3B+, one SHA-256 operation on a 128-byte block takes ≈ 0.02 ms. Six extra hashes add at most ≈ 0.12 ms to the AS’s Phase 2 verification time — well within the noise of network latency on a multi-hop IoT network.

7.3 Storage Overhead

- **GW (per AS):** +2 hash values ($PID_{AS}^{\text{curr}}, PID_{AS}^{\text{old}}$) plus +2 nonces ($\eta_{\text{curr}}, \eta_{\text{old}}$) = +128 bytes.
- **AS (per device):** No change (device state unchanged). Own pseudonym state: +2 hash values = +64 bytes total at AS.
- **Device:** PID_{AS}^{curr} replaces ID_{AS} ; both are 256-bit values. Net change: zero.

The storage cost is concentrated at the GW, which is a capable node and can afford it. Device storage is unchanged.

8 Epoch Management Policy

The epoch rotation period T_{epoch} governs the unlinkability–efficiency trade-off.

Short epochs ($T \approx 1$ hour). Strong unlinkability: each session window contains only $\sim n_s$ sessions where n_s is the session rate per hour. Cost: frequent GW→AS epoch rotation messages; higher likelihood of desync for rarely-active devices.

Long epochs ($T \approx 1$ week). Minimal rotation overhead; low desync risk. Cost: an adversary observing a week of traffic can correlate all sessions to the same AS.

Recommended policy. Set T_{epoch} equal to the expected device sleep cycle multiplied by a security margin of 2–4:

$$T_{\text{epoch}} = 4 \times T_{\text{sleep}}$$

For typical IoT deployments with 15-minute sleep cycles, $T_{\text{epoch}} = 1$ hour. This ensures that:

1. A sleeping device waking up after one sleep cycle will not have missed more than one epoch rotation (given the margin).
2. The adversary observing a single epoch window sees at most $\lfloor T_{\text{epoch}}/T_{\text{sleep}} \rfloor = 4$ linkable sessions per AS.

For deployments where devices can sleep for extended periods, the GW should record the epoch at which each device last enrolled or re-synchronized, and proactively send a re-enrollment invitation if a device’s stored epoch is more than two epochs stale.

9 Discussion

9.1 Comparison with Device Pseudonym Mechanism

Table 9 contrasts the device and AS pseudonym mechanisms.

Table 9: Comparison of device and AS pseudonym mechanisms.

Property	Device pseudonym (PID_{curr})	AS pseudonym ($PID_{\text{AS}}^{\text{curr}}$)
Rotation trigger	Every successful session (Phase 3)	GW-initiated epoch rotation
Rotation authority	AS (computes PID_{new})	GW (generates $\eta^{(k)}$)
De-anonymiser	AS knows ID_D	GW knows ID_{AS}
Desync mechanism	$(m_{\text{curr}}, m_{\text{old}})$ dual-state at AS	$(PID_{\text{AS}}^{\text{curr}}, PID_{\text{AS}}^{\text{old}})$ dual-state at AS and GW
Update transmission	m_H carries m_{new} encrypted	Ψ carries $PID_{\text{AS}}^{\text{next}}$ encrypted
Wire overhead added	None (replaces ID_D)	+256 bits (Ψ field)

The key structural difference is that device pseudonyms rotate on *every session* (driven by the key-exchange protocol), while AS pseudonyms rotate on *epochs* (driven by the GW’s policy). This is appropriate because:

- Device anonymity requires session-level unlinkability: an adversary observing consecutive packets should not link them to the same device.
- AS anonymity requires epoch-level unlinkability: an adversary observing traffic over a day/week should not identify which nodes are ASes. Within an epoch, the AS identity is stable to allow efficient GW-side lookup.

9.2 Complete Anonymity: Who Knows Whom

After both extensions (device pseudonym from the original scheme + AS pseudonym from this document), Table 10 summarises identity knowledge.

Table 10: Identity knowledge matrix after full extension.

Observer	Knows ID_D ?	Knows ID_{AS} ?	Knows ID_{GW} ?
Passive eavesdropper	No (PID_{curr} only)	No (PID_{AS}^{curr} only)	GW is the trust anchor; identity assumed known
AS	Yes (enrolled)	Yes (own identity)	No (knows K_{GW-AS} only)
GW	No (sees only PID_{curr})	Yes (lookup table)	N/A

The GW remains the single privileged de-anonymiser for both devices (via the T_{Acc} accumulator and Phase 3 decryption) and ASes (via the pseudonym lookup table). All other entities and all passive observers have only pseudonymous handles.

9.3 Open Questions for Future Formalisation

1. **ProVerif queries for AS anonymity:** A formal query `query attacker(IDAS)` should be added to verify that the Dolev–Yao adversary cannot derive ID_{AS} from observed messages.
2. **Intra-epoch unlinkability:** Within an epoch, sessions carry the same PID_{AS}^{curr} . Formal unlinkability under a chosen-session attack model should be verified.
3. **Epoch rotation authentication:** The GW-to-AS epoch rotation message (delivering $PID_{AS}^{(k)}$) uses the existing secure channel. This channel’s authentication mechanism should be formally modelled to ensure an adversary cannot inject a false epoch rotation.
4. **Multi-AS scenario:** If a device migrates to a new AS (e.g., due to GW re-assignment), a new enrollment sub-protocol is needed. The interaction between device pseudonym state and AS pseudonym state during migration is a non-trivial design problem.

10 Conclusion

This document has presented a concrete scheme extension that closes the AS identity leakage present in the proposed lightweight IoT authentication scheme. The core idea — replacing ID_{AS} with an epoch-bound GW-managed pseudonym PID_{AS} — is simple,

requires no new cryptographic primitives, and adds only one hash-and-XOR operation on the device side and one 256-bit field (Ψ) to the AS-to-Device message.

The extension achieves AS anonymity and epoch-level unlinkability against passive adversaries while preserving full accountability at the GW. A dual-state ($PID_{AS}^{\text{curr}}, PID_{AS}^{\text{old}}$) mechanism handles epoch-transition desynchronisation in all four possible desync scenarios without requiring re-enrollment.

The overhead is modest: +256 bits of communication per authentication round, $\sim +6$ hash operations in the worst case, and +128 bytes of GW storage per AS. These costs are negligible relative to the security gain.

The design is ready for ProVerif formalisation and prototype implementation as a natural continuation of the current MTP work.

References

- [1] S. Das and M. Khatua, “Lightweight and decoupled distributed authentication for multihop IoT networks,” in *Proc. IEEE COMSNETS*, 2026.
- [2] IEEE, *IEEE Standard for Wireless Access in Vehicular Environments – Security Services for Applications and Management Messages*, IEEE Std. 1609.2-2022, 2022.
- [3] M. Khodaei, H. Jin, and P. Papadimitratos, “Scaling pseudonymous authentication for large mobile systems,” in *Proc. ACM WiSec*. ACM, 2018, pp. 174–184.
- [4] 3GPP, “Security architecture and procedures for 5g system,” 3rd Generation Partnership Project, Tech. Rep. TS 33.501 v18.3.0, 2024.
- [5] A. Pfitzmann and M. Hansen, “A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management,” TU Dresden, Tech. Rep., 2010.
- [6] H. Ali and I. Ahmed, “LAAKA: A lightweight anonymous authentication and key agreement scheme for IoT-fog environment,” *Computers & Security*, vol. 140, p. 103770, 2024.
- [7] X. Zhou, S. Wang, K. Wen, B. Hu, X. Tan, and Q. Xie, “Security-enhanced lightweight and anonymity-preserving user authentication scheme for IoT-based healthcare,” *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 9718–9731, 2024.
- [8] D. He, Y. Cai, S. Zhu, Z. Zhao, S. Chan, and M. Guizani, “A lightweight authentication and key exchange protocol with anonymity for IoT,” *IEEE Trans. Wireless Communications*, vol. 22, no. 11, pp. 7862–7872, 2023.

- [9] O. Alruwaili, M. Tanveer, S. A. Aldossari, S. Alanazi, and A. Armghan, “RAAF-MEC: Reliable and anonymous authentication framework for IoT-enabled mobile edge computing environment,” *Internet of Things*, vol. 29, p. 101459, 2025.